DEPI · DATA ENGINEERING GRADUATION PROJECT · PROJECT 07

AI FOR FEEDBACK

AI-Enhanced Customer Feedback Data Pipeline

PROJECT DOCUMENTATION

Part 1: Project Planning & Management

Part 2: Literature Review

Part 3: Requirements Gathering

PROJECT TEAM

TEAM MEMBER	ROLE
Omar Samir Abdelnabi Sharaf	Team Leader • Project coordination & integration
Sayed Badawy Farouk Ahmed	Data ingestion & text preprocessing
Omar Mahmoud Ahmed Omar Abdelbaq	AI / NLP inference & evaluation
Youssef Mohammed Ezzat	Database architecture & data quality
Amr Yasser Elhabashy	Airflow orchestration & analytics dashboard

DOCUMENT DETAILS	
Program	Digital Egypt Pioneers Initiative (DEPI) – Data Engineering Graduation Project
Repository	https://github.com/sayedelakabawy/depi-final-project
Document version	1.0 – October 2026
Final project delivery	Last week of December 2026 (exact day to be confirmed)

Table of Contents

Executive Summary . . . 3

Proposed project name . . . 3

Document scope . . . 3

The project in brief . . . 3

Alignment with the project brief . . . 3

1. Project Planning & Management . . . 5

1.1 Project concept and objectives . . . 5

1.2 Proposed technical approach . . . 6

1.3 Work plan and delivery schedule . . . 7

1.4 Team management and responsibilities . . . 8

1.5 Risk management and contingency plan . . . 9

1.6 Success measurement and acceptance . . . 10

2. Literature Review . . . 12

2.1 Research approach . . . 12

2.2 Overview of similar solutions . . . 12

2.3 Comparison with our proposed project . . . 14

2.4 Identified gap . . . 14

2.5 How we leveraged these findings . . . 15

2.6 References . . . 15

3. Requirements Gathering . . . 16

3.1 Overview & scope . . . 16

3.2 Stakeholder analysis & needs . . . 16

3.3 Functional requirements . . . 16

3.4 Non-functional requirements . . . 17

3.5 Data requirements and proposed schema . . . 18

3.6 Data quality and error-handling rules . . . 18

3.7 Assumptions and constraints . . . 18

3.8 Proposed test scenarios . . . 19

Executive Summary

Proposed project name

AI FOR FEEDBACK – *AI-Enhanced Customer Feedback Data Pipeline* (Project 07, DEPI Data Engineering Graduation Project).

Document scope

This document is the project documentation for the AI FOR FEEDBACK graduation project. It sets out the plan, the research background and the specification against which the pipeline will be built and accepted, and is organised in three parts:

- **Part 1 – Project Planning & Management:** concept and objectives, technical approach, work plan, team responsibilities, risks and success measures.
- **Part 2 – Literature Review:** four comparable solutions assessed against the same criteria, the gap they leave, and how our design builds on them.
- **Part 3 – Requirements Gathering:** stakeholders, functional and non-functional requirements, data schema, quality rules and test scenarios.

Implementation evidence (source code, Airflow runs, dashboards, evaluation results) will be added as each milestone is completed. Technical allocations, tools, dataset and performance targets are proposals until confirmed by the team and the instructor.

The project in brief

ASPECT	SUMMARY
Problem	Companies receive thousands of customer reviews and support comments as unstructured text. Reading them manually is slow, and raw feedback contains duplicates, missing values and inconsistent formatting.
Solution	A scheduled, AI-powered ETL pipeline: CSV ingestion → validation, cleaning and deduplication → Hugging Face sentiment inference → PostgreSQL storage → SQL/Pandas summaries and a dashboard, all orchestrated by Apache Airflow.
Key outputs	Raw and enriched datasets, per-record sentiment with score, model ID and processing timestamp, an error table, aggregated views by product, category, date and sentiment, and an analytics dashboard.
Optional extension	LLM/LangChain extraction of structured fields (product, issue_type, complaint, requested_action) from free-form feedback – pursued only after the mandatory scope is complete.
Timeframe	19 October – late December 2026 (approx. 10 weeks).

Alignment with the project brief

The table maps each deliverable required by the project brief to where it is specified in this document and the milestone at which it will be delivered.

BRIEF DELIVERABLE	SPECIFIED IN	MILESTONE
Raw-data ingestion	§1.2, FR-1.1 – FR-1.2	M1
Text-processing module	§1.2, FR-1.3 – FR-1.4, §3.6	M1
AI transformation module	§1.2, FR-2.1 – FR-2.5	M2

BRIEF DELIVERABLE	SPECIFIED IN	MILESTONE
Database (raw, enriched, errors, runs)	§1.2.3, FR-3.1, §3.5	M3
Airflow DAG	§1.2.1, FR-4.1	M4
Data-quality checks	§3.6, FR-4.2	M1 – M4
Sentiment dataset	§3.5, FR-2.2 – FR-2.4	M2 – M3
Analytical visualization	§1.6, FR-3.2 – FR-3.3	M4
Architecture diagram	§1.2.1 (Figure 1.1)	Included
README	§1.6.2, NFR-07	M5

1. Project Planning & Management

PROJECT AT A GLANCE Design a scheduled AI-powered ETL pipeline that turns unstructured customer reviews into traceable sentiment records, SQL-ready summaries, and an analytics dashboard.

Project profile

FIELD	DETAILS
Program	Digital Egypt Pioneers Initiative (DEPI) – Data Engineering Graduation Project
Project title	AI FOR FEEDBACK | AI-Enhanced Customer Feedback Data Pipeline
Timeframe	19 October – late December 2026 (approx. 10 weeks)
Submission	Last week of December 2026 – exact date to be confirmed
Repository	github.com/sayedelakabawy/depi-final-project (verify shareable URL and access before submission)

Team composition and proposed allocations are given in Section 1.4. Technical allocations (other than Team Leader), tools, dataset and performance targets are proposed and subject to team and instructor confirmation.

1.1 Project concept and objectives

1.1.1 Problem statement

Companies receive large volumes of unstructured reviews and customer support comments. Reading these records manually is slow and makes it difficult to detect overall satisfaction, recurring complaints, or product-level trends. Raw feedback also contains duplicates, missing values, inconsistent formatting, and entries that AI models may not process reliably.

The team therefore needs a dependable data pipeline that preserves the raw customer message, produces clear sentiment classifications, and stores outputs in a structured database for repeatable analysis.

1.1.2 Proposed solution

SOLUTION Batch CSV ingestion → Data validation → Text cleaning / deduplication → Hugging Face sentiment inference → PostgreSQL storage → SQL / Pandas aggregation → Dashboard, all orchestrated and monitored by Apache Airflow.

1.1.3 Project objectives

1. Collect a documented CSV dataset of customer feedback; keep API/database ingestion as a possible future extension.
2. Check required columns and dates, retain original text, create a normalized text field, and reject or quarantine invalid rows.
3. Use a suitable pre-trained Hugging Face model to predict Positive, Neutral, or Negative sentiment, storing the model score and identifier.
4. Preserve raw and enriched feedback separately, including processing timestamps, batch/run metadata, and explicit inference failures.
5. Automate the ETL workflow with Airflow, with logs, quality gates, retry handling, and safe re-runs.

6. Provide sentiment trends and summary counts by product, category, date, and sentiment through SQL/Pandas and a dashboard.

1.1.4 Scope and boundaries

IN SCOPE FOR DECEMBER DELIVERY	OUT OF SCOPE / FUTURE EXTENSION
Batch processing of permitted CSV reviews; validation, normalization, deduplication.	Real-time customer-chat monitoring and live event streaming.
A pre-trained 3-class sentiment classifier; qualitative and quantitative evaluation.	Training a large language model from scratch or guaranteeing multilingual support.
PostgreSQL raw/enriched/error datasets and repeatable SQL analyses.	Enterprise-grade scaling, cloud production deployment, or high availability.
Airflow scheduling, quality rules, error logging, dashboard, documentation.	Optional extension: LLM/LangChain structured extraction (product, issue_type, complaint, requested_action); automated support-ticket action and agent-based response writing.

END-USER BENEFIT Business and product analysts can explore trends and negative-feedback hotspots without having to read every review manually.

1.2 Proposed technical approach

1.2.1 System architecture

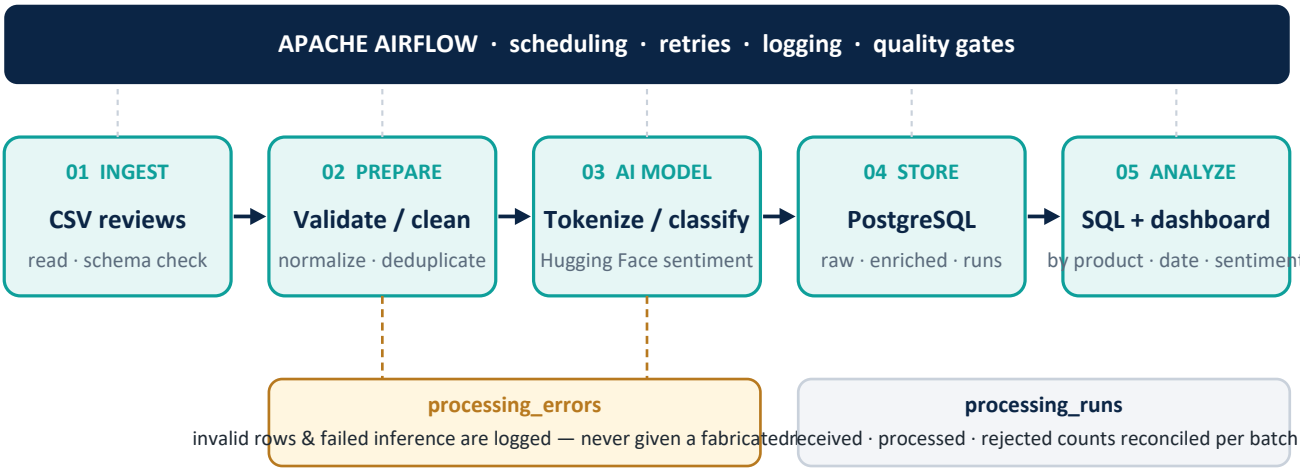


Figure 1.1 – End-to-end architecture of the AI FOR FEEDBACK pipeline

Apache Airflow manages task order, scheduled runs, retries and monitoring across the entire flow. Invalid or failed records are logged rather than silently assigned a sentiment.

1.2.2 Technology stack and rationale

LAYER	PROPOSED TECHNOLOGY	WHY WE USE IT
Ingestion & cleaning	Python, Pandas	Portable CSV ingestion, validation, text normalization and deduplication.
AI inference	Hugging Face Transformers	Reusable pre-trained model; labels/scores captured with model metadata.
Persistent storage	PostgreSQL + SQL	Reliable structured tables, constraints, joins and aggregate views.

LAYER	PROPOSED TECHNOLOGY	WHY WE USE IT
Orchestration	Apache Airflow	Dependency management, batch scheduling, reruns and execution logs.
Reporting	Power BI or Matplotlib	Product-, date- and sentiment-based visual summaries.
Collaboration	Git / GitHub	Pull requests, team issue board, documentation and review history.

1.2.3 Proposed data contract and storage

The final schema will be validated against the selected dataset. Suggested raw fields include `source_review_id`, `original_text`, `review_date`, and optional product/category metadata. The transformed layer adds `cleaned_text`, `sentiment`, model score, `model_id`, `processed_at`, `processing_status`, and a run identifier.

DATASET / TABLE	ROLE	PRIMARY CONTROL
raw_feedback	Unmodified ingested reviews and source metadata.	Preserve originals; track source IDs / batches.
enriched_feedback	Clean text and AI sentiment results.	Unique source keys, model ID, timestamp.
processing_runs	Batch counts, duration and run statuses.	Reconcile received, processed and rejected counts.
processing_errors	Invalid records and model/database failures.	Reason, source ID, attempt/run ID, no secrets.

> **DESIGN DECISION** A reported model score is not automatically a calibrated probability. Verify that the model supports all three required classes, and document unsupported-language behavior before choosing it.

1.3 Work plan and delivery schedule

1.3.1 Timeline: October → December 2026

The team plans to submit during the final week of December 2026; the precise day is not yet confirmed. To protect that deadline, core development should finish by mid-December, followed by integration, testing, documentation and final demo preparation before year-end. All dates below are proposed planning windows, not official deadlines.

PERIOD	WORK PACKAGE	TANGIBLE CHECKPOINT
19–31 Oct 2026	Kickoff; confirm dataset permissions, requirements, roles, repository, architecture and quality rules.	M0: Agreed scope, schema draft, team backlog, sample data.
1–12 Nov 2026	Build CSV ingestion; raw storage; schema checks; empty/invalid/duplicate handling.	M1: Reproducible ingestion and cleaning tests.
13–24 Nov 2026	Select a documented 3-class model; tokenize, benchmark samples and validate label/score output.	M2: AI enrichment module and baseline evaluation report.
25 Nov – 8 Dec	Finish PostgreSQL enriched/error tables, idempotent loads and SQL summaries; start cross-module integration.	M3: Queryable raw/enriched records and source-to-target reconciliation.
9–17 Dec 2026	Build and test Airflow DAG, retries, execution logs, quality gates and dashboard.	M4: Complete scheduled ETL run with dashboard evidence.
18–23 Dec 2026	Run end-to-end tests, verify failure recovery, resolve bugs, freeze scope, finalize README and screenshots.	M5: Feature freeze and acceptance-test evidence.
24–30 Dec 2026	Final acceptance review, presentation rehearsal, documentation packaging, team sign-off and submission.	M6: Final repository, report, screenshots and demo ready.

1.3.2 Milestone gates

GATE	ACCEPTANCE EVIDENCE / EXIT CONDITION
M1 – Clean data	Same sample CSV produces stable counts; invalid/duplicate records are accounted for.
M2 – AI inference	Valid reviews receive label, score (if supported), model ID and timestamp; failures are logged.
M3 – Storage	Raw and enriched records are queryable; repeat processing does not duplicate final records.
M4 – Automation	Airflow completes extraction → transform → inference → load → quality check; logs are visible.
M5 – Feature freeze	Dashboard queries work, key negative/duplicate/failure tests pass, and setup steps are documented.
M6 – Dec submission	Final PDF/report, README, source repository, architecture graphic and demo evidence approved.

SCHEDULE SAFEGUARD Freeze optional features by 17 December. Keep 31 December as contingency only, if the confirmed submission deadline permits; the team should aim to submit by 30 December.

1.4 Team management and responsibilities

1.4.1 Proposed responsibilities – 5 members

Omar Samir Abdelnabi Sharaf is identified as Team Leader; all detailed technical allocations remain proposals for team approval.

TEAM MEMBER / ROLE	KEY RESPONSIBILITIES	EXPECTED WORK OUTPUT
Omar Samir Abdelnabi Sharaf Team Leader / Integration	Own scope, timeline and project decisions; review architecture; coordinate merges, end-to-end integration and submission.	Project plan, issue board, milestone reviews, integrated build, final demo.
Sayed Badawy Farouk Ahmed Data Ingestion / Preprocessing	Select permitted dataset; implement CSV reader, schema validation, text cleanup, deduplication and batch statistics.	Ingestion scripts, documented data contract, cleaning tests, sample dataset.
Omar Mahmoud Ahmed Omar Abdelbaq AI / NLP Engineer	Select/evaluate a suitable 3-class model, tokenize, run inference, record model metadata and handle AI failures.	Inference module, evaluation report, model usage documentation.
Youssef Mohammed Ezzat Database / Data Quality	Design PostgreSQL raw/enriched/error tables; enforce unique keys and build checks, SQL summaries and query views.	Schema/ERD, SQL scripts, data quality queries and reports.
Amr Yasser Elhabashy Airflow / Visualization	Design Airflow DAG, retries and logs; connect pipeline to SQL views and build final trend dashboard.	Airflow DAG, scheduling evidence, dashboard and screenshots.

1.4.2 Collaboration, reporting and accountability

- **Weekly checkpoint:** each member reports completed tasks, current blockers, next deliverable and confidence in milestone date.
- **Shared GitHub Issues board:** To Do → In Progress → In Review → Done. Each item has an owner, acceptance rule and due window.
- **Feature branches and pull requests:** review changes before merging; protect the stable main branch where practical.
- **Shared conventions:** agree early on naming conventions, input/output contracts, environment configuration and error codes.

- **Escalation:** for blockers over 48 hours, notify the team lead; reassign tasks or switch to the agreed backup plan rather than waiting for the next meeting.
- **Decision log:** keep a short record of changes affecting scope, dataset, selected model, team responsibilities or December delivery.

1.4.3 Dependencies and handoffs

> **CRITICAL PATH** Approved dataset + schema → Validated cleaning → Three-class model integration → Database persistence → Airflow orchestration → Dashboard + quality evidence → late-December presentation.

1.5 Risk management and contingency plan

1.5.1 Risk register

Risk assessment is qualitative; likelihood and impact should be updated during weekly checkpoints.

RISK (OWNER)	LEVEL	PREVENTION / CONTINGENCY
Dataset unavailable / unclear license Sayed	Medium / High	Check permitted usage in October; keep a documented open/synthetic fallback CSV.
Mixed-language, sarcasm or weak sentiment accuracy Omar Mahmoud	High / Medium	Choose model based on actual language; test a labeled subset; document limitations and error patterns.
Model has only 2 classes; Neutral missing Omar Mahmoud	Medium / High	Confirm label mapping and three-class support before integration; select another documented model if needed.
Slow inference / memory constraints Omar Mahmoud + Omar Samir	Medium / Medium	Benchmark early; use manageable batches, input truncation within model limits and documented hardware.
Duplicate records on Airflow retry / rerun Youssef + Amr	Medium / High	Use stable IDs, database constraints and idempotent upserts; test the same batch twice.
Airflow/database environment fails Amr + Youssef	Medium / High	Pin versions, test small DAG first, keep separate executable modules and backup clean dataset.
Privacy or secrets leak in repository All members	Low / High	Use public/permitted/synthetic datasets; never commit credentials; redact personal info in logs and screenshots.
Delayed integration close to deadline Omar Samir	Medium / High	Start module integration in November; freeze new features by 17 December; defer nonessential enhancements.

Level is expressed as likelihood / impact.

1.5.2 Incident management approach

- **Detect:** make failures visible through Airflow status, run counts and an error/rejection table.
- **Triage:** identify whether the failure is ingestion, schema validation, AI model, database, or orchestration.
- **Recover:** retry only safe tasks; preserve raw input; use idempotent inserts/upserts to avoid extra rows.
- **Document:** capture root cause, fix, evidence, owner and next prevention action in an Issue.

1.5.3 Resource and cost assumptions

Plan for an educational, batch-scale dataset using local development hardware and open-source Python, PostgreSQL, Hugging Face libraries, and Apache Airflow. Use Power BI or Matplotlib depending on reliable access. No cloud expense is assumed; if hardware or storage is insufficient, reduce the batch size and document measured

throughput.

> **SCOPE CHANGE CONTROL** Optional LLM extraction, APIs, streaming, Docker and cloud deployment are future extensions; they will not displace mandatory ETL, AI sentiment, database, dashboard and quality evidence.

1.6 Success measurement and acceptance

1.6.1 Proposed KPIs and validation

These are project targets, not achieved results. Actual performance thresholds and the reference sentiment labels must be agreed after selecting a dataset and model.

MEASURE	PROPOSED TARGET / DEFINITION	HOW TO VERIFY
Data integrity	0 duplicate source IDs in curated data; every accepted row retains original and cleaned text.	SQL uniqueness and field audits.
Record traceability	100% of source rows reconciled as enriched, rejected, failed or pending.	Per-batch input/output/error count check.
AI output completeness	All successful rows have a predicted 3-class label, model ID and processing timestamp; store returned score.	Database NOT NULL / range checks; model output audit.
Model evaluation	Publish confusion matrix, accuracy and macro-F1 on an agreed held-out labeled sample; target agreed after baseline.	Reproducible evaluation notebook/report.
Pipeline reliability	Demonstrate 5 successful consecutive controlled end-to-end batch runs; failures log visibly.	Airflow history plus error scenarios.
Idempotent recovery	Re-running the same batch creates no extra curated records.	Two-run row-count comparison.
Analytics readiness	Working summaries/visuals by product, category, date and sentiment wherever source fields exist.	SQL outputs + filter demonstration.
Reproducibility	Another teammate can run the sample workflow from the README on the documented environment.	Fresh-setup walkthrough and screenshots.

1.6.2 December submission checklist

DELIVERABLE	EVIDENCE EXPECTED
Source code + GitHub	Modular ingestion, cleaning, AI, storage and Airflow DAG; dependencies, .env example and README.
Data + DB schema	Permitted input CSV, example enriched sentiment output, PostgreSQL schema/ERD and SQL aggregate views.
AI + quality proof	Model ID, sentiment evaluation, schema/deduplication tests, error examples, row reconciliation.
Automation + analytics	Successful Airflow DAG screenshots, dashboard screenshots and sample SQL metrics.
Documentation + demo	Architecture diagram, setup/run guide, known limitations, presentation and test scenario evidence.

1.6.3 Formal acceptance criteria

1. The team can run a test batch from raw CSV to final analytical output without manually classifying individual reviews.
2. Empty, invalid, duplicate and inference-failed records have explicit outcomes; unclassified failures are never given fabricated predictions.

3. The database retains original feedback and processing metadata, and a repeated batch does not inflate final counts.
4. An evaluator can inspect the Airflow run, query the sentiment tables, navigate the dashboard and repeat the setup from the documentation.

2. Literature Review

2.1 Research approach

Before fixing the design of the **AI-Enhanced Customer Feedback Data Pipeline**, the team searched for existing projects and production solutions that turn unstructured customer text into analysable data. Sources included GitHub repositories, vendor reference architectures (AWS, Astronomer), and peer-reviewed or preprint research on sentiment and aspect-based analysis. Each solution was assessed against the same criteria:

- **Orchestration:** how the workflow is scheduled, monitored and retried.
- **AI enrichment:** which model is used and what it outputs per record.
- **Depth of insight:** sentiment only, or also aspects, issues and requested actions.
- **Storage & analytics:** where raw and enriched data live and how business users query them.
- **Cost & dependency:** reliance on paid APIs, specific clouds, or credentials.
- **Data quality & fault tolerance:** validation, invalid-row handling, and inference failures.

Four representative solutions were selected, covering an open-source academic pipeline, a managed cloud solution, an orchestration vendor reference, and a large-scale industry deployment.

2.2 Overview of similar solutions

2.2.1 Solution A — Amazon Reviews Sentiment Analysis Pipeline (MLOps-Group-3, open source)

An open-source MLOps project that classifies Amazon product reviews (UCSD Amazon Reviews 2023 dataset) as positive, neutral or negative. It uses modular **Apache Airflow** DAGs for acquisition, sampling, validation, preprocessing and labelling; trains **BERT/RoBERTa** classifiers tracked in **MLflow**; versions data with **DVC**; deploys on **Google Cloud** (Cloud Storage, Vertex AI); and adds a RAG component (**OpenAI + Pinecone**) that writes aspect-wise review summaries shown in **Streamlit** dashboards [1].

Advantages	Limitations
• **Modular orchestration:** separate Airflow DAGs for each stage make the pipeline easy to rerun and debug. • **Validation with logging:** invalid rows are detected and logged before modelling. • **Strong MLOps practice:** data versioning (DVC), experiment tracking (MLflow), CI/CD and model serving. • **Context-aware models:** transformer classifiers outperform lexicon-based sentiment on complex text.	• **Heavy, paid stack:** depends on GCP, Vertex AI, OpenAI and Pinecone credentials, which raises cost, setup effort and vendor lock-in. • **Insights are not per-record fields:** aspects appear only inside generated summaries; complaint type or requested action are not stored as queryable columns. • **Model-centric, not analytics-centric:** outputs live in cloud buckets and a vector store rather than a relational schema with ready business aggregations. • **Limited reported evaluation:** the README lists features but reports no accuracy or benchmark figures.

2.2.2 Solution B — Sentiment at Scale with Amazon Comprehend, AWS Glue and Amazon Athena (AWS)

An AWS reference walkthrough in which a serverless **AWS Glue** (Apache Spark) job reads Amazon product reviews from **S3**, cleans and batches the text, calls the **Amazon Comprehend** batch sentiment API, joins results back to the reviews, and writes Parquet partitioned by product category. A Glue Crawler registers the table so it can be queried with **Amazon Athena** (SQL) and visualised in **Amazon QuickSight** [2].

Advantages	Limitations
• **Fully managed and scalable:** no servers or model hosting to maintain. • **Analytics-ready output:** partitioned Parquet + Data Catalog lets analysts query results with SQL immediately. • **Clean separation** of raw and processed data in storage layers.	• **Sentiment only:** no extraction of products, issue types or requested actions in this design. • **Vendor lock-in and per-request cost:** every stage is an AWS service and Comprehend is billed per unit of text. • **Black-box model:** the sentiment model cannot be inspected, versioned or swapped by the team. • **No explicit orchestration or failure handling** for failed inference batches is described.

2.2.3 Solution C — Customer Feedback Analysis with Cohere and OpenSearch in Airflow (Astronomer)

A reference use case from Astronomer (the company behind Astro, a managed Airflow platform). An Airflow DAG ingests (synthetic) customer feedback from an API into **OpenSearch**, filters relevant records, and uses **dynamic task mapping** to call **Cohere** in parallel for a sentiment label + confidence and a text embedding per review. Results are written back to each document and a k-NN search finds positive testimonials similar to a target text [3].

Advantages	Limitations
• **Parallel inference in Airflow:** dynamic task mapping scales model calls per record. • **Label and confidence stored with each record,** which supports auditing. • **Semantic search** via embeddings enables finding similar feedback.	• **Paid external API:** every record requires a Cohere API call and key. • **Synthetic demo data** and a search-engine store rather than a relational analytical model. • **No aggregated summaries** by product, category or date, and no explicit data-quality gates.

2.2.4 Solution D — End-to-End Aspect-Guided Review Summarization at Scale (Wayfair, 2025)

An industry system described by Wayfair researchers. An LLM (Gemini 1.5 Flash) processes every review individually and returns **JSON** with up to five product aspects, each labelled positive, mixed or negative. Rare aspects are mapped to canonical terms (reducing ~178,000 distinct aspects to ~19,000), then the top aspects and sampled reviews are used to generate product-page summaries. The released dataset covers about 11.8 million reviews for 92,000 products, and an online A/B test was run on ~493,000 products [4].

Advantages	Limitations
• **Structured per-review extraction:** LLM output is constrained to a JSON schema, producing queryable aspect-level data. • **Vocabulary consolidation:** mapping aspects to canonical names keeps categories consistent over time. • **Proven at production scale** with measured business impact.	• **Commercial LLM dependency:** quality depends strongly on the chosen LLM and on API cost. • **Goal is customer-facing summaries,** not internal complaint tracking; it does not extract requested actions or route issues. • **Weaker on rare products or specialised terms,** as the authors acknowledge; the full production pipeline is not open source.

2.2.5 Supporting research

Two research foundations informed the design. First, **Aspect-Based Sentiment Analysis (ABSA)**, formalised in the SemEval-2014 shared task, showed that a single review often contains different sentiments toward different aspects (e.g. good product, late delivery), so a single overall label hides the actual problem [5]. Second, the **Hugging Face Transformers** library made pre-trained models such as DistilBERT easy to load and run locally, removing the need for paid inference APIs for the core sentiment step [6][7]. Pipeline design best practices (idempotent tasks, retries, task-level logging) follow standard Airflow guidance [8].

2.3 Comparison with our proposed project

Table 2.1 compares the reviewed solutions with our pipeline across the evaluation criteria.

Criterion	A: Amazon Reviews (MLOps-Group-3)	B: AWS Comprehend + Glue + Athena	C: Astronomer Cohere + OpenSearch	D: Wayfair Aspect Summarization	Our project
Orchestration	Apache Airflow DAGs	AWS Glue job (serverless Spark)	Apache Airflow DAG with dynamic task mapping	Internal production pipeline	Apache Airflow DAG with retries and per-record failure handling
AI enrichment	Fine-tuned BERT/RoBERTa + OpenAI (RAG)	Amazon Comprehend (managed API)	Cohere API (classification + embeddings)	Gemini 1.5 Flash LLM	Pre-trained Hugging Face model run locally; optional LLM/LangChain extraction layer
Output per record	Sentiment label	Sentiment label	Sentiment label, confidence, embedding	Up to 5 aspects with sentiment (JSON)	Sentiment, confidence score, model_id, processed_at; optional structured fields (product, issue_type, complaint, requested_action)
Actionable insight	Aspect summaries (free text)	None	Similar-testimonial search	Product-page summaries	SQL-queryable sentiment by product, category and date; issue type and requested action as columns in the optional extension
Storage	GCP buckets + Pinecone vector DB	S3 Parquet + Glue Data Catalog	OpenSearch index	Not public	Relational SQL database holding raw and enriched records
Analytics layer	Streamlit dashboards	Athena SQL + QuickSight	None (logs only)	None for internal analysis	SQL/Pandas aggregates by product, category, date, sentiment; views for BI tools
Data quality	Validation DAG + invalid-row logs	Basic filtering in ETL	Not described	Not described	Quality gates after ingestion and transformation; duplicate detection
Cost & dependency	High (GCP, OpenAI, Pinecone)	Pay-per-use; AWS lock-in	Paid API key required	Paid LLM API	Low: open-source stack, runs locally/on any cloud

Table 2.1 — Comparison of reviewed solutions with the proposed pipeline

2.4 Identified gap

Across the reviewed work, solutions either (a) produce only an overall sentiment label that is easy to aggregate but does not explain **why** customers are unhappy (B, C), or (b) extract richer aspects but store them as free-text summaries or depend on costly cloud and LLM services (A, D). None of them combines, in one low-cost, reproducible pipeline: data-quality validation, local transformer-based sentiment with confidence scores, preservation of the original text, ready-made SQL summaries for business teams and, as an extension, structured complaint fields. This is the gap our project targets.

2.5 How we leveraged these findings

What we adopted

- Treating AI inference as a **transformation step inside an Airflow DAG**, preceded by validation and cleaning (from A and C).
- **Logging invalid rows** instead of failing the whole run (from A).
- Storing the **confidence score with each prediction** for auditability (from C).
- Writing results in an **analytics-ready layout** that analysts can query with SQL (from B).
- For the optional extraction layer: forcing LLM output into a **fixed JSON schema** and mapping values to a **controlled vocabulary** (from D).

How we improved on them

- **Lower cost and no lock-in:** the core sentiment model is an open Hugging Face model running locally, so no paid API is needed for the main pipeline.
- **Actionable structured fields (optional extension):** product, issue_type, complaint and requested_action are planned as columns, not hidden inside summaries.
- **Dual-state storage:** raw and enriched records are kept side by side in a relational database for auditing and reprocessing.
- **Fault tolerance:** failed or timed-out inference is captured per record without stopping the batch.
- **Business-ready outputs:** automated SQL/Pandas summaries by product, category, date and sentiment feed dashboards directly.

2.6 References

[1] MLOps-Group-3. (2024). *Amazon Reviews Sentiment Analysis* [Source code]. GitHub. https://github.com/MLOps-Group-3/Amazon-Reviews-Sentiment-Analysis

[2] Hasson, R. (2018, June 4). How to scale sentiment analysis using Amazon Comprehend, AWS Glue and Amazon Athena. *AWS Machine Learning Blog*. https://aws.amazon.com/blogs/machine-learning/how-to-scale-sentiment-analysis-using-amazon-comprehend-aws-glue-and-amazon-athena/

[3] Astronomer. (n.d.). Use Cohere and OpenSearch to analyze customer feedback in an MLOps pipeline. *Astronomer Documentation*. https://www.astronomer.io/docs/airflow/2.x/use-case-llm-customer-feedback

[4] Boytsov, I., Eusden, C., Rochat, M.-C., Pierson, M., DeGenova, V., Balyasin, M., & Walt, J. (2025). End-to-end aspect-guided review summarization at scale. *arXiv preprint* arXiv:2509.26103. https://arxiv.org/abs/2509.26103

[5] Pontiki, M., Galanis, D., Pavlopoulos, J., Papageorgiou, H., Androutsopoulos, I., & Manandhar, S. (2014). SemEval-2014 Task 4: Aspect based sentiment analysis. In *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)* (pp. 27–35). ACL.

[6] Wolf, T., et al. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations* (pp. 38–45). ACL.

[7] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint* arXiv:1910.01108.

[8] Harenslak, B., & de Ruiter, J. (2021). *Data Pipelines with Apache Airflow*. Manning Publications.

All online sources were accessed on 19 October 2026.

3. Requirements Gathering

3.1 Overview & scope

This section defines the specification for the **AI-Enhanced Customer Feedback Data Pipeline**. The system ingests unstructured customer reviews and support comments, validates and cleans the text, enriches the records using a pre-trained Hugging Face sentiment analysis model, orchestrates the entire workflow via Apache Airflow, and persists structured data for downstream analytics and visualizations.

3.2 Stakeholder analysis & needs

STAKEHOLDER	NEEDS / EXPECTED VALUE
Business or product analysts	Clean, structured sentiment data categorized by product and date; queryable database schemas for ad-hoc SQL analysis and reporting; scheduled data updates via Airflow orchestration.
Customer support team	High-level analytical dashboards and summary metrics; clear tracking of customer pain points and product feature feedback over time.
Product managers	Trend summaries by product, category, and date to identify possible areas for improvement.
Data engineering / project team	A reproducible pipeline, clear logs, data-quality controls, documented interfaces, and maintainable code.
Project evaluator / instructor	Clear evidence of the ETL process, AI integration, orchestration, storage, quality checks, and analytical output.

3.3 Functional requirements

Module 1: Ingestion & Text Processing

ID	REQUIREMENT	PRIORITY
FR-1.1	The system shall ingest customer feedback from a documented CSV schema for the first release.	Must
FR-1.2	The system shall validate required columns and identify missing IDs, empty text, invalid dates, and malformed rows.	Must
FR-1.3	The system shall preserve the original feedback and create a cleaned/normalized text field without overwriting the source text.	Must
FR-1.4	The system shall detect duplicate records using a documented stable key or duplicate rule.	Must

Module 2: AI Inference & Sentiment Enrichment

ID	REQUIREMENT	PRIORITY
FR-2.1	**Pre-trained model integration:** the pipeline shall use a pre-trained Hugging Face Transformer model (e.g. a RoBERTa-based sentiment classifier). The selected model must support all three required classes; binary models do not provide a Neutral class.	Must
FR-2.2	**Sentiment classification:** the AI module shall assign a sentiment label (Positive, Neutral, Negative) to each valid feedback entry.	Must
FR-2.3	**Confidence score:** the inference step shall extract and store the score returned by the model with each predicted label; its meaning shall be documented (see NFR-08).	Must

ID	REQUIREMENT	PRIORITY
FR-2.4	**Processing metadata:** every processed record shall carry a processing timestamp, model identifier and pipeline execution metadata.	Must
FR-2.5	**Exception handling in inference:** in case of model inference timeouts or memory failures, the system shall capture the error gracefully without halting the entire batch.	Must

Module 3: Storage & Data Aggregation

ID	REQUIREMENT	PRIORITY
FR-3.1	**Dual-state persistence:** the database shall store both raw customer records and enriched records (with sentiment, scores, and timestamps) for auditing purposes.	Must
FR-3.2	**Multi-dimensional aggregation:** using SQL/Pandas, the system shall generate summary metrics grouped by product / service, category / topic, date / time range, and sentiment type.	Must
FR-3.3	**Analytical views:** the system shall expose aggregated data views optimized for analytical dashboard tools (e.g. Power BI, Tableau, Streamlit, or Matplotlib/Seaborn).	Must

Module 4: Orchestration & Data Quality

ID	REQUIREMENT	PRIORITY
FR-4.1	**Airflow DAG orchestration:** the complete ETL workflow shall be scheduled, executed, and monitored as an Apache Airflow Directed Acyclic Graph (DAG).	Must
FR-4.2	**Data quality checks:** quality gates shall run post-ingestion and post-transformation to verify schema consistency, missing values, and record counts.	Must

3.4 Non-functional requirements

ID	REQUIREMENT
NFR-01 – Reliability	A failed record or task must be logged with enough information to investigate it; retry behavior must be defined.
NFR-02 – Data integrity	Unique keys and database constraints should prevent duplicate storage and enforce required fields where applicable.
NFR-03 – Performance	The pipeline should process the agreed test dataset within a target duration defined after measuring a baseline on the team's hardware.
NFR-04 – Maintainability	Code should be split into clear modules, use configuration rather than hard-coded secrets, and include dependency/version instructions.
NFR-05 – Reproducibility	The same documented input and configuration should produce consistent schema and workflow behavior, allowing for model/runtime variation where relevant.
NFR-06 – Security & privacy	Credentials must not be committed to GitHub. The team should prefer public/synthetic data and avoid exposing personal data in logs or screenshots.
NFR-07 – Usability	A team member unfamiliar with the project should be able to follow the README to run a small test and interpret the output.
NFR-08 – Transparency	The output should identify the model used and distinguish predicted labels from verified ground-truth sentiment; the score must be documented as returned by the model, not assumed to be a calibrated probability.

3.5 Data requirements and proposed schema

The exact schema depends on the chosen dataset. The following is a proposed minimum design; fields such as product and category are optional if the source does not provide them.

FIELD	TYPE / EXAMPLE	PURPOSE
source_review_id	Text / 105	Stable source identifier; unique when available.
product	Text / Product A	Supports product-level summaries if supplied.
category	Text / Delivery	Source category, if available.
original_text	Text	Original feedback preserved as received.
cleaned_text	Text	Text after documented normalization.
sentiment	Text / POSITIVE	Predicted sentiment label returned by the selected model.
sentiment_score	Numeric / 0.97	Model score as returned; interpretation depends on model.
model_id	Text	Exact model identifier or version.
processed_at	Timestamp	Time when the record was processed.
processing_status	Text / success or failed	Supports tracking and failure investigation.

A separate processing log or error table may store failure reason, source ID, attempt number, and task/run identifier. Avoid storing secrets or unnecessary personal information. The meaning of a sentiment score must be documented from the selected model; do not automatically describe every score as a calibrated probability.

3.6 Data quality and error-handling rules

1. Reject or quarantine records with missing required identifiers or empty feedback, and log a reason.
2. Preserve original text; apply normalization to a separate field. Cleaning should not remove meaning-bearing negation or punctuation without a clear reason.
3. Define duplicate handling explicitly, preferably using a stable source ID; if none exists, document the chosen combination of fields.
4. Use database primary/unique constraints and an idempotent upsert strategy where appropriate.
5. If inference fails, record the failure and retry according to policy. Do not silently assign a sentiment label to a failed record.
6. Track counts for rows received, invalid rows, duplicates, successfully enriched rows, and failed rows so that the pipeline can be reconciled.

3.7 Assumptions and constraints

- The first release uses batch processing, not real-time streaming.
- CSV is the initial input source; API/database ingestion is a possible extension.
- The team has access to a machine or environment capable of running Python, PostgreSQL, Airflow, and the selected model.
- The initial sentiment model and input language will be selected after checking model documentation and sample results.
- The visualization will be built using a tool the team can operate reliably; Power BI or Matplotlib are both possible options.
- The team will use public, permitted, or synthetic data and will check dataset/model licenses before distribution.

3.8 Proposed test scenarios

TEST	EXPECTED BEHAVIOR
Valid review	Record is cleaned, analyzed, and stored with a sentiment label and score if returned.
Empty review	Record is rejected or quarantined with a reason; no fabricated sentiment is stored.
Duplicate source ID	Duplicate is detected and does not create an extra enriched record.
Model unavailable	Failure is logged; configured retry/failure behavior is visible in Airflow.
Database unavailable	Task fails visibly and can be retried after the connection is restored.
Same input rerun	Existing records are updated or skipped according to the chosen idempotency rule.
Missing product/category	Pipeline handles null values without crashing; summaries document how missing categories are treated.
Mixed-language or unsupported input	Behavior is tested and documented; unsupported language is not assumed to be accurately classified.